Clustering and Initialization: An Experimental Study of K-Means and K-Means++

Fatemeh Moayedirad

June 2026

Abstract

This report studies the effect of initialization in K-Means clustering. Standard K-Means with random initialization is compared with K-Means++ initialization, using implementations written from scratch in Python. The experiments use synthetic Gaussian blob datasets and the real-world Digits dataset. The results show that K-Means++ improves stability and convergence on the synthetic datasets, especially by reducing the risk of poor local minima. On the Digits dataset, however, the results are more nuanced: random initialization obtains better average inertia and Adjusted Rand Index, while K-Means++ converges in slightly fewer iterations. This shows that careful initialization is useful, but it does not guarantee better clustering on every dataset.

1 Introduction and Methodology

Clustering is an unsupervised learning task where the goal is to group data points according to similarity, without using predefined labels. K-Means is a common clustering algorithm because it is simple, fast, and interpretable. It represents each cluster by a centroid and assigns each point to the nearest centroid.

The main weakness of K-Means is its dependence on the initial centroids. Since the objective is non-convex, different initializations can lead to different local minima, different final inertia values, and different convergence behavior. K-Means++ was introduced by Arthur and Vassilvitskii [1] to reduce this sensitivity. It selects the first centroid uniformly at random from the data, and each next centroid is sampled with probability proportional to the squared distance from the closest already selected centroid. This is known as D^2-sampling and encourages the initial centers to be spread across the dataset.

After initialization, both methods use the same K-Means optimization procedure: they alternate between assigning each point to the nearest centroid and updating each centroid as the mean of its assigned points. Therefore, the comparison focuses specifically on the effect of the initialization strategy.

The objective minimized by K-Means is the final inertia:

$$J = \sum_{i=1}^{n} \min_{1 \le j \le k} \|x_i - c_j\|^2.$$

Lower inertia means that points are closer to their assigned centroids, but it does not necessarily mean better agreement with true labels. For this reason, the Digits experiment also uses Adjusted Rand Index (ARI) as an external evaluation metric.

2 Implementation and Experimental Setup

Both algorithms were implemented from scratch in Python using NumPy. No library implementation of K-Means was used for model fitting. Scikit-learn was used only for dataset generation/loading and for computing ARI. The project code is organized into `src/kmeans.py` for standard K-Means, `src/kmeanspp.py` for K-Means++ initialization, and `src/utils.py` for plotting, standardization, and repeated-run summaries. All experiments are run from `notebooks/experiments.ipynb` with fixed random seeds.

The maximum number of iterations was set to 300 and the convergence tolerance to 10^{-4}. If an empty cluster occurred during the centroid update step, its centroid was reinitialized using a farthest-point strategy, so that the algorithm could continue without losing a cluster.

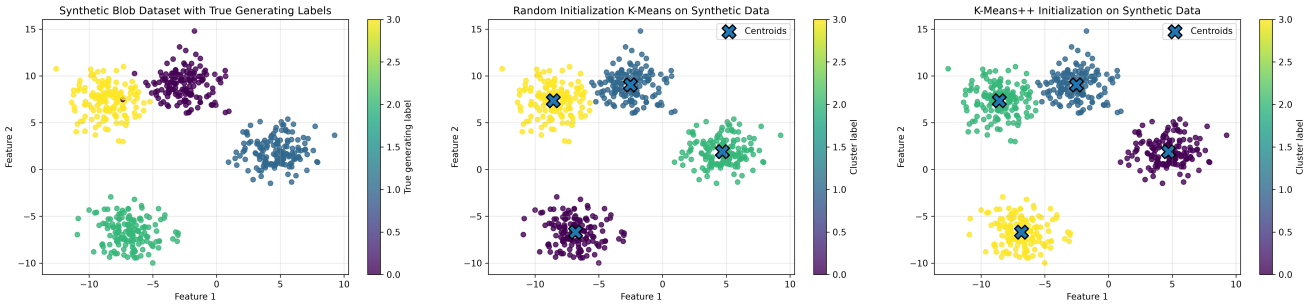
Two datasets were used. The first is a two-dimensional synthetic blob dataset with 600 samples, 4 clusters, and controllable cluster overlap through `cluster_std`. This dataset is useful for visualization and controlled experiments. The second is the Digits dataset from `sklearn.datasets`, with 1797 samples, 64 features, and 10 classes. It is a small real-world high-dimensional multiclass dataset. The Digits features were standardized before clustering because K-Means is based on Euclidean distance.

No train/test split was used because the task is unsupervised clustering rather than supervised prediction. Labels were not used for fitting; on the Digits dataset, true labels were used only after clustering to compute ARI. Each main experiment was repeated 30 times with different random seeds to compare average behavior and variability. The full per-run results and summary tables were exported as CSV files in the project repository, while the report includes compact summary tables for readability.

3 Results

3.1 Synthetic Dataset: Visual Comparison

The first experiment uses the two-dimensional synthetic blob dataset. Figure 1 compares the true generating labels with the cluster assignments obtained by random K-Means and K-Means++.



(a) True labels **(b)** Random initialization **(c)** K-Means++ initialization

Figure 1: Visual comparison on the synthetic blob dataset. The black crosses represent final centroids. Cluster colors are arbitrary and are used only as group identifiers.

Figure 1(a) shows four Gaussian-like groups. Figures 1(b) and 1(c) show that both methods identify the four main groups in this representative run, and the centroids are located near the visible cluster centers. This means that random initialization can still find a good solution when the data has a clear cluster structure. However, this single visual comparison does not measure robustness, because a different random initialization may lead to a worse local minimum.

3.2 Synthetic Dataset: Convergence and Stability

Figure 2 compares the convergence curves for one representative run. K-Means++ starts from a much lower inertia, meaning that its initial centroids are already closer to a good configuration. Both methods reach the same final inertia, 2587.789, but random initialization needs 6 iterations while K-Means++ reaches the same value in 3 iterations. In this run, the advantage of K-Means++ is therefore faster convergence, not a better final objective value.

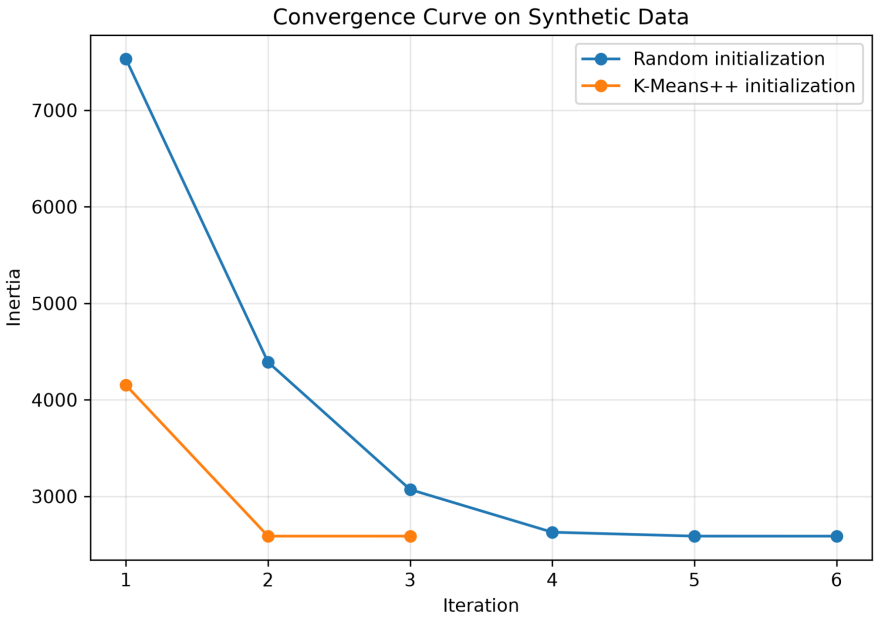
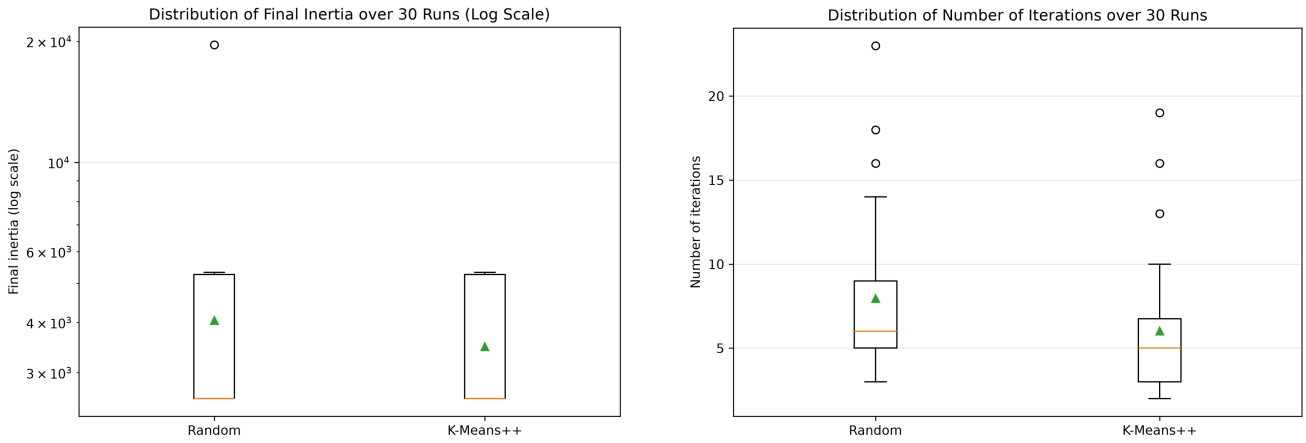


Figure 2: Convergence curves on the synthetic dataset. K-Means++ starts from a lower inertia and reaches the final solution in fewer iterations.

To evaluate stability more reliably, both methods were then run 30 times on the synthetic dataset with `cluster_std = 1.5`. Figure 3(a) shows the distribution of final inertia on a logarithmic scale, and Figure 3(b) shows the distribution of the number of iterations.



(a) Final inertia, log scale

(b) Number of iterations

Figure 3: Stability comparison over 30 runs on the synthetic dataset with `cluster_std = 1.5`. The triangle marker indicates the mean.

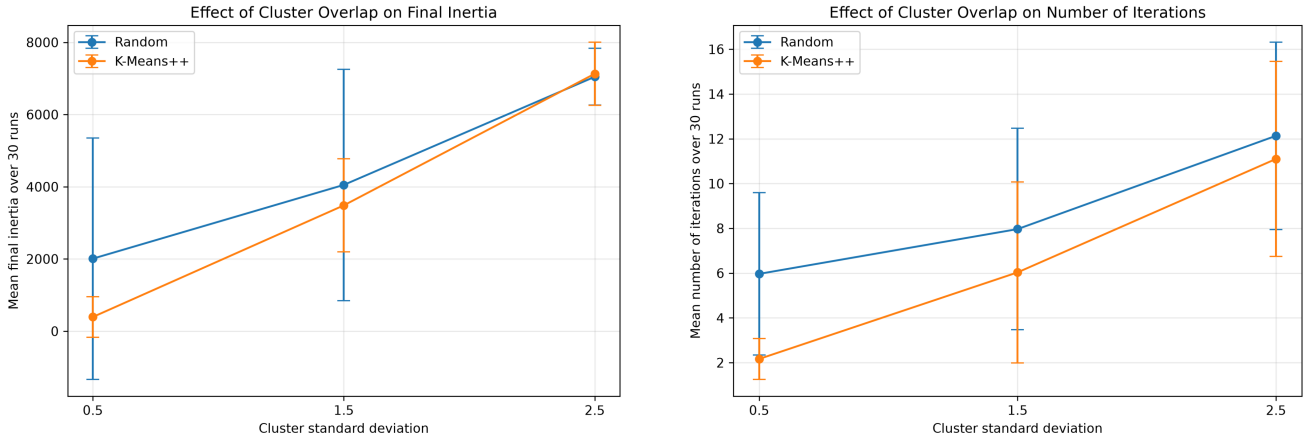
The lower parts of the inertia distributions are close, which means that both methods often reach similar good solutions. The difference is in variability: random initialization has a much larger spread and a very large outlier, showing that some runs converge to poor local minima. K-Means++ has lower mean inertia and smaller variability. Numerically, the mean inertia is 3484.626 for K-Means++ and 4051.584 for random initialization, while the standard deviation is 1290.080 for K-Means++ and 3204.605 for random initialization. K-Means++ also needs fewer iterations on average, 6.033 compared with 7.967. This supports the conclusion that K-Means++ is more stable on this synthetic dataset. Table 1 reports the corresponding CSV summary in compact form.

Table 1: Summary of the synthetic stability experiment over 30 runs with `cluster_std = 1.5`.

Method	Mean inertia	Std. inertia	Min inertia	Max inertia	Mean iter.	Std. iter.
K-Means++	3484.626	1290.080	2587.789	5325.708	6.033	4.038
Random	4051.584	3204.605	2587.789	19610.710	7.967	4.499

3.3 Effect of Cluster Overlap

The next experiment changes the cluster standard deviation to study how overlap affects performance. Figure 4 shows that increasing `cluster_std` makes the task harder for both methods.



(a) Mean final inertia

(b) Mean number of iterations

Figure 4: Effect of cluster overlap on performance over 30 runs. Error bars represent one standard deviation and are shown only as variability indicators.

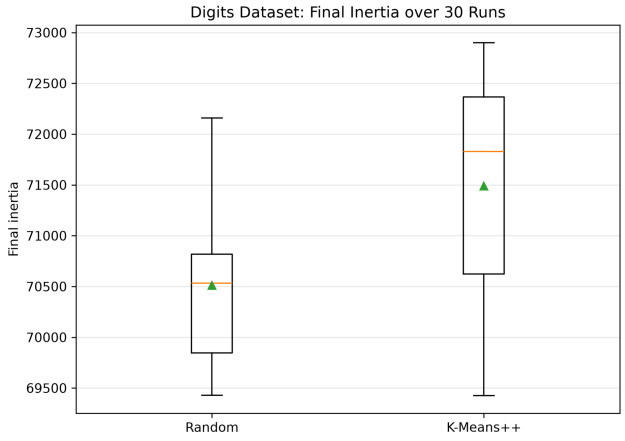
In Figure 4(a), mean inertia increases as clusters become more dispersed and overlapping. For `cluster_std = 0.5`, K-Means++ has a clear advantage: the mean inertia is 393.809 compared with 2009.222 for random initialization. At `cluster_std = 1.5`, K-Means++ still performs better on average, but the gap is smaller. At `cluster_std = 2.5`, the two methods become very similar; random initialization even obtains a slightly lower mean inertia, although the difference is small. Figure 4(b) shows a similar trend for convergence speed: both methods require more iterations when overlap increases, while K-Means++ remains slightly faster. Overall, K-Means++ helps most when there is still meaningful cluster structure to exploit; when clusters become highly overlapping, the data structure itself becomes the dominant difficulty. Table 2 gives the compact numerical summary exported from the experiment.

Table 2: Summary of the cluster-overlap experiment over 30 runs.

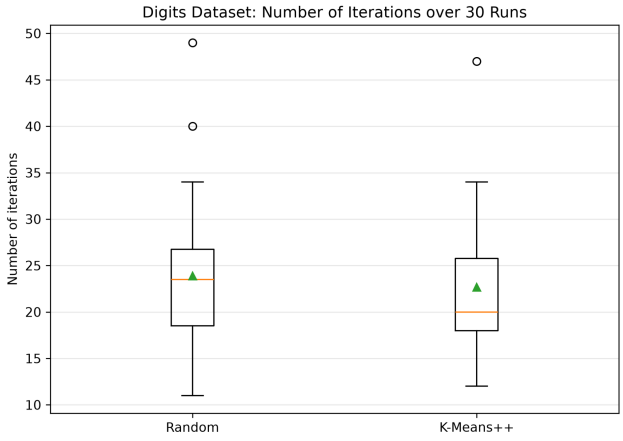
`cluster_std`	Method	Mean inertia	Std. inertia	Mean iter.	Std. iter.
0.5	K-Means++	393.809	564.698	2.167	0.913
0.5	Random	2009.222	3344.082	5.967	3.624
1.5	K-Means++	3484.626	1290.080	6.033	4.038
1.5	Random	4051.584	3204.605	7.967	4.499
2.5	K-Means++	7129.700	875.816	11.100	4.358
2.5	Random	7046.114	785.854	12.133	4.183

3.4 Digits Dataset: Quantitative Results

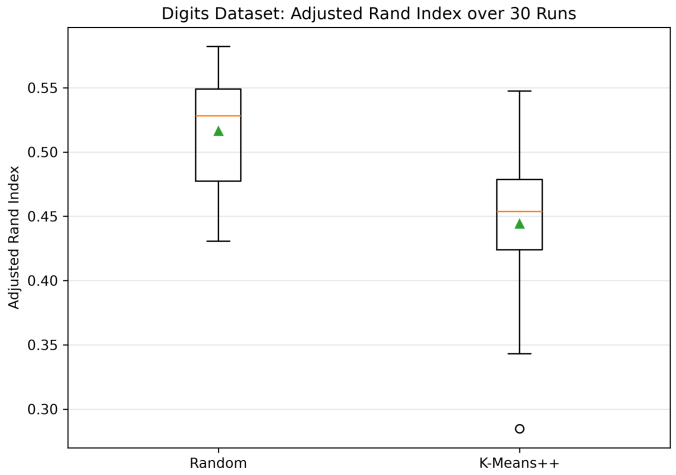
Figure 5 shows the results on the Digits dataset. This dataset is more challenging than the synthetic blobs because it is high-dimensional and its classes are not necessarily compact spherical clusters in Euclidean space.



(a) Final inertia



(b) Number of iterations



(c) Adjusted Rand Index

Figure 5: Results on the Digits dataset over 30 runs. The triangle marker indicates the mean across runs.

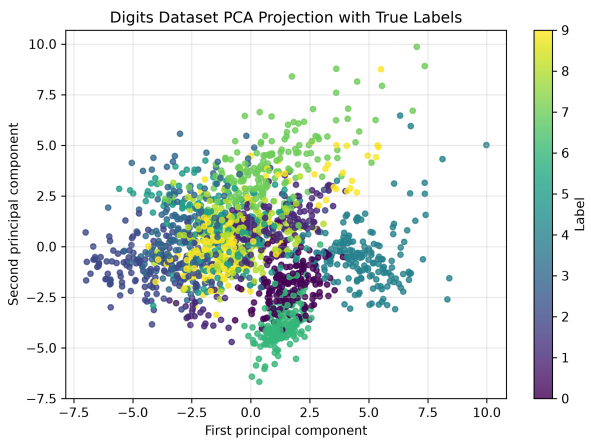
On this dataset, random initialization obtains lower mean inertia: 70511.098 compared with 71491.330 for K-Means++. Random initialization also achieves higher mean ARI, 0.516 compared with 0.444 for K-Means++. K-Means++ still has a small advantage in convergence speed, with 22.667 mean iterations compared with 23.900 for random initialization, but this difference is small. These results show an important limitation: K-Means++ improves initialization, but it does not guarantee better final clustering on every dataset. Table 3 reports the corresponding numerical summary.

Table 3: Summary of the Digits dataset experiment over 30 runs.

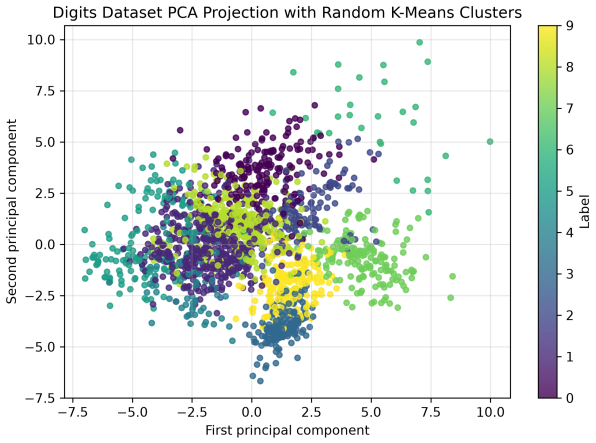
Method	Mean inertia	Std. inertia	Mean iter.	Std. iter.	Mean ARI	Std. ARI
K-Means++	71491.330	1017.540	22.667	7.448	0.444	0.059
Random	70511.098	809.157	23.900	8.036	0.516	0.040

3.5 Digits Dataset: PCA Visualization

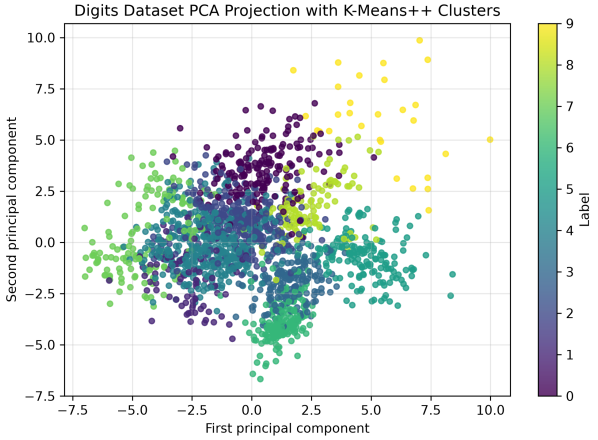
Figure 6 provides a qualitative visualization of the Digits dataset using the first two principal components. Clustering was performed on the original standardized 64-dimensional data; PCA was used only for visualization.



(a) True digit labels



(b) Random initialization clusters



(c) K-Means++ clusters

Figure 6: Two-dimensional PCA visualization of the Digits dataset. PCA is used only for visualization; in panels (b) and (c), the colorbars denote arbitrary cluster labels rather than true digit labels.

Figure 6(a) shows that the true digit labels overlap substantially in the first two principal components. Figures 6(b) and 6(c) show that both methods produce some coherent regions, but the learned clusters do not align perfectly with the true digit labels. The colorbars in these two panels should be read as arbitrary cluster indices, not as digit identities. This explains why the ARI values are only moderate. The main limitation is not only initialization; it is also the mismatch between the K-Means objective, which favors compact Euclidean clusters, and the more complex structure of digit images.

4 Conclusion

The experiments show that K-Means++ is useful but not universally superior. On synthetic blob datasets, K-Means++ improves stability, reduces the risk of poor local minima, and often converges faster. Its benefit is strongest when the data has a meaningful cluster structure. However, on the Digits dataset, random initialization achieves better average inertia and ARI, while K-Means++ only converges slightly faster. This suggests that initialization matters, but it cannot fully overcome the limitations of the K-Means objective on high-dimensional, non-spherical real-world data. Overall, the comparison across datasets shows that the advantage of K-Means++ is dataset-dependent: it is clear on structured synthetic blobs, but less consistent on the high-dimensional Digits dataset.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged

within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

References

[1] D. Arthur and S. Vassilvitskii. *k-means++: The Advantages of Careful Seeding*. Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007.